

# Mini-Exposé: Kontexttransparenz für Coding Agents

Inspizierbares, graphbasiertes Kontext-Engineering und seine Wirkung auf Effizienz und Entwicklerkontrolle  
Ausschreibung: „Mehr Effizienz mit Coding Agents? Kontexttransparenz und Kontext-Engineering“

---

## 1. Warum: das Problem

Bei Coding Agents (Claude Code, Cursor) entscheidet die **Kontextbereitstellung** über Erfolg oder Halluzination (Gloaguen et al. 2026). Zwei Probleme bleiben offen:

- **Ineffizient.** Agents explorieren Codebases reaktiv und rufen systematisch mehr Kontext ab als sie nutzen (ContextBench, 2026). Statische Kontext-Dateien (`CLAUDE.md`) helfen kaum: handgeschrieben nur ~4 % bessere Lösungsrate, generiert sogar schlechter, beide > 20 % teurer (Gloaguen et al. 2026).
- **Intransparent.** Löst der Agent falsch, sieht der Entwickler nicht, *welcher Kontext zu welcher Entscheidung* führte; Kontext ist eine Black Box. Werkzeuge setzen erst *während* der Ausführung an (AgentStepper). Ob man die Kontextauswahl **vor** dem Start bewerten und korrigieren kann und ob das die Qualität verbessert, ist bislang kaum systematisch untersucht, obwohl Entwickler diese präventive Kontrolle nachweislich suchen (Dhanorkar et al. 2026).

## 2. Was wird Technisch umgesetzt:

Ein MCP-Server, der den Kontext für eine Aufgabe nicht nur automatisch assembliert, sondern auch die Auswahl vor der Agentenausführung inspizier- und korrigierbar macht. Eine Codebase wird als Wissensgraph (Neo4j, Tree-Sitter für Python/TypeScript, lokale Embeddings) repräsentiert. Es gibt drei Funktionen.

1. **Context Assembly (hybrid).** Seed-Knoten über semantische Embedding-Ähnlichkeit *und* exakte Namens-/Pfad-Treffer; Expansion per Graph-Traversal (strukturell, historisch über Git/Issues, testbasiert) zu einem aufgabenspezifischen Kontext-Block.
2. **Entscheidungspfad.** Pro Knoten eine nachvollziehbare Begründung: Score und Query-Token-Überlappung für Seeds, konkrete Graph-Kante und Traversal-Kette für den Rest.
3. **Inspektions- und Korrektur-Interface.** Ein browserbasiertes Review-UI zeigt den geplanten Teilgraphen samt Retrieval-Trace. Der Entwickler sieht *warum* jeder Knoten drin ist, kann Knoten entfernen/ergänzen, die Anfrage editieren und neu abfeuern: Kontext als editierbarer Vorschlag statt Black Box.

Der eigentliche Beitrag ist nicht der Graph (Stand der Technik), sondern die Retrieval-Transparenz als Eingriffspunkt sowie deren empirische Messung im Feld. Reines Embedding-Retrieval liefert lediglich eine Rangliste. Die explizite Graphstruktur begründet die Auswahl und macht sie korrigierbar.

## 3. Forschungsfragen

- **HF1 (Effizienz):** Verbessert graph-assemblierter Kontext die Effizienz (Iterationen, Dauer, Erstkorrektheit) gegenüber reaktiver Exploration, statischer `CLAUDE.md` und manuell ausgewählten Snippets?
- **HF2 (Transparenz):** Hilft ein expliziter Entscheidungspfad Entwicklern, fehlerhafte Kontextauswahlen *vor* dem Lauf zu erkennen?
- **HF3 (Intervention, explorativ):** Führt die Korrektur der Kontextauswahl vor der Ausführung zu besseren Agent-Ergebnissen?

## 4. Wie: Methodik

Design Science Research mit iterativen Build-Evaluate-Zyklen, zwei Studien: **Studie 1 (HF1)** vergleicht vier Kontextbedingungen (B1 nur Aufgabe, B2 `CLAUDE.md`, B3 manuelle Snippets, B4

Graph) über Bugfix-, Refactoring- und Feature-Aufgaben; Metriken werden automatisch geloggt. **Studie 2 (HF2/HF3)** ist eine kontrollierte Studie am Review-Interface mit gezielt verfälschten Kontextauswahlen (seeded errors). **Feldumgebung:** aktiv entwickelte Projekte (Python/TypeScript), Pilot ist ein realer Webshop; der Wert von Transparenz ist auf eingefrorenen Benchmark-Snapshots prinzipiell nicht messbar.

**Umsetzung und Stand (Juni 2026):** Die vollständige Umsetzung des Artefakts ist konstruktiver Bestandteil der Arbeit. Ein lauffähiger Prototyp (Parser, Graph, hybride Assembly-Pipeline, MCP-Server *und* ein erstes interaktives Review-Interface) belegt bereits die Machbarkeit; eine erste Eval zeigt brauchbare Kontextabdeckung. Im Verlauf der Arbeit werden Artefakt und Interface zur Studienreife ausgebaut und anschließend empirisch untersucht.